



Tally: Non-Intrusive Performance Isolation for Concurrent Deep Learning Workloads

Wei Zhao[†]
Stanford University
Stanford, USA
CentML
Toronto, Canada
wzhao@cs.stanford.edu

Anand Jayarajan
University of Toronto
Toronto, Canada
Vector Institute
Toronto, Canada
CentML
Toronto, Canada
anandj@cs.toronto.edu

Gennady Pekhimenko
University of Toronto
Toronto, Canada
Vector Institute
Toronto, Canada
CentML
Toronto, Canada
pekhimenko@cs.toronto.edu

Abstract

GPU underutilization is a significant concern in many production deep learning clusters, leading to prolonged job queues and increased operational expenses. A promising solution to this inefficiency is GPU sharing, which improves resource utilization by allowing multiple workloads to execute concurrently on a single GPU. However, deploying GPU sharing in production settings faces critical obstacles due to the limitations of existing mechanisms, including high integration costs, inadequate performance isolation, and limited application compatibility. To address these issues, we introduce *Tally*, a non-intrusive GPU sharing mechanism that provides robust performance isolation and comprehensive workload compatibility. The key to *Tally*'s robust performance isolation capability lies in its fine-grained thread-block-level GPU kernel scheduling strategy, which allows the system to effectively mitigate interference caused by workload co-execution. We evaluate *Tally* on a diverse range of workloads and show that it incurs an average overhead of only 7.2% on the 99th-percentile latency of high-priority inference tasks when executed concurrently with best-effort training workloads, compared to 188.9% overhead exhibited by the state-of-the-art GPU sharing systems like TGS, while achieving over 80% of TGS's system throughput.

CCS Concepts: • **Computing methodologies** → **Concurrent algorithms**; Artificial intelligence; • **Computer systems organization** → *Parallel architectures*.

Keywords: GPU sharing, performance isolation, cloud infrastructure, deep learning, systems for machine learning

[†]Part of the work done as a student at the University of Toronto.



This work is licensed under a Creative Commons Attribution International 4.0 License.

ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0698-1/25/03

<https://doi.org/10.1145/3669940.3707282>

ACM Reference Format:

Wei Zhao, Anand Jayarajan, and Gennady Pekhimenko. 2025. Tally: Non-Intrusive Performance Isolation for Concurrent Deep Learning Workloads. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*, March 30–April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3669940.3707282>

1 INTRODUCTION

Deep learning (DL) has demonstrated astonishing performance across a wide range of applications, including image recognition [29, 33, 60], machine translation [7, 22, 35, 59], recommendation systems [3, 76], and autonomous driving [15, 73]. Recent advances in generative AI [11, 54], exemplified by the development of large language models (LLMs) such as ChatGPT [47], have further elevated the influence of DL to unprecedented levels. To facilitate the research and deployment of DL-based applications, many organizations [34, 56, 67] are investing in building computing infrastructures with large fleets of GPU devices. For example, Meta has built two data center-scale clusters for AI, each equipped with hundreds of thousands of NVIDIA H100 GPUs with an estimated cost as high as half a billion dollars [24, 34]. As the demand for computational resources continues to escalate, efficient utilization of GPUs becomes paramount to mitigate costs and ensure scalability for AI applications.

Nevertheless, studies have noted significant GPU underutilization in large-scale DL clusters. For example, a recent study by Alibaba revealed that the median GPU utilization in their production DL cluster was only 4.2% [67]. This inefficiency arises from the need for production clusters to accommodate a diverse range of workloads, many of which are unable to fully utilize the capacity of GPUs. Training workloads, for instance, may underutilize GPUs due to bottlenecks in CPU execution such as stalls caused by data fetching and preprocessing [6]. On the other hand, the utilization of on-line inference tasks is intrinsically tied to real-time request rates, which are subject to substantial fluctuations causing unpredictable GPU idleness [38]. For example, Microsoft's production trace [77] has revealed significant burstiness in

user requests, with frequent spikes in demand up to 50× the average. This makes usage prediction and resource allocation challenging for inference workloads, resulting in resource over-provisioning to ensure high quality of service [38, 74], which exacerbates the underutilization problem.

To enhance the efficiency of DL clusters, recent studies [31, 41, 70, 71] suggest GPU sharing as a promising solution. This approach aims to enhance hardware utilization by allowing multiple low-utilization tasks to share a single GPU’s resources. The same Alibaba study conducted a simulation on their production traces and demonstrated that an effective GPU sharing mechanism could reduce the amount of GPU resources required by the cluster by 50% on average, with reductions reaching up to 73% during peak demand periods. These findings underscore the potential of GPU sharing to mitigate GPU underutilization and significantly improve the efficiency of large-scale DL clusters.

Despite these promising results, GPU sharing has not yet achieved widespread adoption in production. One major obstacle is the lack of robust performance isolation mechanisms in most current GPU sharing systems. Many workloads in DL clusters are governed by service-level agreements (SLAs) with strict performance requirements, often measured in milliseconds [18]. These workloads are commonly referred to as *high-priority* or *latency-critical* tasks. In contrast, workloads that can tolerate extended processing times are known as *low-priority* or *best-effort* tasks. While it seems logical to pack best-efforts tasks along with high-priority tasks to utilize potential GPU idle time, this approach can cause significant interference to the performance of high-priority workloads, violating their SLAs. Our experiments on state-of-the-art GPU sharing mechanisms reveal that such interference can cause up to a 20× slowdown in tail latency (see Section 5), making them impractical for production.

Another hindrance to the broader adoption of GPU sharing lies in the intrusive nature of many existing solutions [4, 13, 14, 70, 71, 79]. These systems often necessitate extensive modifications to user code or the underlying DL frameworks (e.g., PyTorch [49], TensorFlow [2]). Such requirements not only greatly limit users’ flexibility in developing workloads but also impose substantial burdens on users and cluster providers for workload adaptation and framework upgrades, leading to considerable integration and maintenance costs [31]. Moreover, many of these solutions require the workloads to adhere to specific characteristics, such as ensuring GPU kernels used by the workload are always idempotent [28]. These constraints significantly limit their applicability in production clusters, which typically host a diverse range of workloads with heterogeneous characteristics.

In light of these observations, we argue that for practical deployment in production DL clusters, a GPU sharing system must meet the following three criteria: (1) it must be *non-intrusive* to seamlessly integrate into existing clusters and DL workloads, (2) provide *performance isolation* guarantees

to ensure that the SLAs of high-priority tasks are effectively maintained during shared execution, and (3) be *generalizable* across a diverse range of DL applications. Building on these objectives, we propose *Tally*, a novel non-intrusive GPU sharing system for large-scale production DL clusters that ensures robust performance isolation for high-priority tasks when co-located with best-effort workloads, while maximizing hardware efficiency and overall system throughput. We design Tally as a transparent virtualization layer between the application level and the GPU. It intercepts GPU kernel launch API calls and employs a task-agnostic scheduling strategy that prioritizes the execution of high-priority kernels while opportunistically scheduling best-effort kernels during GPU idle periods.

To efficiently co-locate high-priority and best-effort tasks on a GPU with performance isolation guarantees, it is essential to schedule best-effort kernels in a controlled manner when the GPU is idle and promptly switch to high-priority kernels when needed. However, modern GPUs lack kernel scheduling primitives to terminate a running kernel and only allow context switching between kernels at the granularity of entire kernels [25, 28]. This limitation introduces significant overhead on high-priority tasks, especially since many best-effort DL workloads involve long-running kernels. To overcome this challenge and enable finer-grained kernel scheduling, Tally uses two block-level kernel scheduling primitives: *slicing* and *preemption*. Slicing divides large kernels into smaller segments, while preemption allows for the interruption of active kernels. Our key insight is that these two scheduling primitives can be implemented in a non-intrusive and task-agnostic manner through a series of GPU kernel transformation passes, leveraging the GPU programming model which organizes kernels as collections of independent thread blocks.

While slicing and preemption are effective mechanisms to manage co-execution interference, each has its trade-offs. Kernel slicing is a lightweight transformation but can incur higher overhead due to multiple kernel launches. In contrast, kernel preemption requires only a single kernel launch but involves a more complex transformation with potential synchronization overhead. Due to these differences, the optimal choice of scheduling primitives and configuration parameters (e.g., the degree of slicing) varies across different kernels. Towards this end, Tally uses a profile-guided resource provisioning strategy in its scheduling algorithm that transparently profiles the performance of best-effort kernels on-the-fly across various scheduling configurations to select the best option that meets the latency requirements of the high-priority task. This approach ensures that interference from best-effort tasks remains within acceptable limits while striving for optimal system throughput.

To evaluate Tally’s ability to provide effective performance isolation, we prepare a comprehensive benchmark suite consisting of six training and six inference workloads that cover

a wide range of popular DL models and frameworks (details shown in table 2). We evaluate the co-located performance of latency-critical inference tasks and best-effort training tasks by simulating the production traffic patterns for the inference tasks using the publicly available Microsoft Azure Function Trace 2021 (MAF2) dataset [77]. Across all the combinations, Tally incurs a mere 7.2% overhead on average in the 99th-percentile latency of inference tasks, compared to 252.3%, 345.0%, 195.5%, and 188.9% exhibited by state-of-the-art non-intrusive GPU sharing solutions, Time-Slicing [25], MPS, MPS-Priority [1], and TGS [68] respectively. At the same time, Tally is able to achieve 80% of the system throughput attained by TGS, demonstrating the effectiveness of Tally in maximizing the GPU efficiency while providing strong performance isolation guarantees.

In summary, we make the following contributions.

- We propose Tally, a non-intrusive GPU sharing solution with robust performance isolation guarantees.
- We introduce a novel approach to achieve fine-grained GPU kernel scheduling in a non-intrusive and task-agnostic manner using kernel slicing and preemption primitives. We further propose a priority-aware kernel scheduler that dynamically selects the appropriate scheduling primitives to optimize GPU throughput while upholding performance isolation guarantees.
- We evaluate Tally on a diverse range of training and inference workloads and show that Tally can achieve system throughput on-par with state-of-the-art GPU sharing solutions while ensuring strong performance isolation.

2 BACKGROUND

Modern large-scale DL clusters are commonly shared among numerous users within an organization and are designed to support a diverse range of workloads [31, 70]. These include training, fine-tuning, and inference tasks for a variety of model architectures, such as convolutional neural networks (CNNs) [29, 50], transformer-based large language models (LLMs) [9, 22, 62], graph neural networks (GNNs) [63, 72], and reinforcement learning (RL) models [55, 57]. These workloads are implemented using various DL frameworks, including widely adopted open-source platforms like PyTorch [49], TensorFlow [2], and JAX [10], as well as proprietary software stacks [67]. Given the diversity and compute-intensive nature of DL workloads, general-purpose GPUs have become the predominant accelerator in DL clusters for their expressive programming model and massive parallel processing capabilities.

GPU architecture and programming model. Despite variations across vendors (e.g. NVIDIA, AMD), the core architecture and programming model of modern GPUs in DL clusters remain largely similar. GPUs are composed of hundreds of parallel processing units called Streaming Multiprocessors

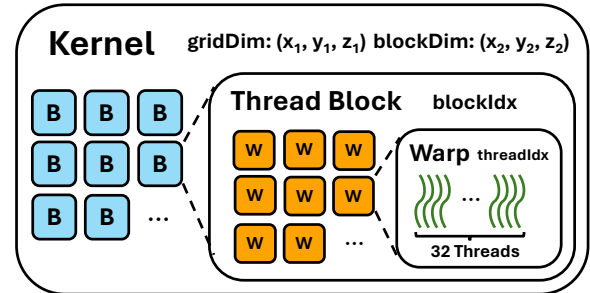


Figure 1. Overview of the GPU programming model.

(SMs). The computational tasks executed on GPUs, known as kernels, are programmed using specialized languages like CUDA [44]. As illustrated in Figure 1, GPU kernels follow a hierarchical thread group structure consisting of threads, warps, thread blocks, and grids. Threads within a kernel identify themselves using built-in variables such as *threadIdx* and *blockIdx* to determine their task assignments. Threads within a warp execute instructions in a SIMT (Single Instruction, Multiple Threads) fashion, where multiple threads execute the same instruction on different pieces of data simultaneously. All warps within a thread-block are scheduled onto a single SM, enabling them to share data using fast on-chip shared memory. Additionally, threads within a block can synchronize their execution using synchronization primitives to coordinate memory accesses. Finally, thread blocks within a grid can be scheduled and executed independently in any order, either sequentially or in parallel [44]. This independent scheduling capability allows thread blocks to scale effectively across multiple SMs.

GPU sharing systems. The diverse nature of DL workloads, coupled with their inherent inefficiencies, has led to significant concerns about GPU underutilization in many production DL clusters [30, 70]. To address this challenge, researchers have proposed numerous cluster-level optimizations [65, 69–71], among which GPU sharing has emerged as a promising approach to improve utilization by consolidating multiple low-utilization jobs onto the same GPU. From an implementation perspective, current GPU sharing solutions can be broadly categorized into two levels: application-level and runtime-level approaches. Application-level solutions typically involve the introduction of new programming interfaces designed for concurrent DL execution. For example, Salus [71] exposes a set of scheduling primitives for defining the iteration boundaries of a training job, enabling the scheduler to manage different training workloads at the iteration-level granularity. Similarly, HFTA [65] provides APIs that enable developers to group multiple hyper-parameter tuning jobs into a single application, facilitating efficient execution on a single GPU.

In contrast to application-level solutions that require modifications to user-level applications, runtime-level GPU sharing solutions operate transparently by integrating directly into either DL frameworks [70] (e.g., PyTorch [49], TensorFlow [2]), container orchestration systems [68] (e.g., Kubernetes [26]), or the GPU driver [1, 28, 46]. For example, NVIDIA GPUs provide two built-in driver-level concurrency mechanisms: Time-Slicing [25] and MPS [1], which enable transparent sharing of GPU resources in temporal and spatial manners respectively. Additionally, NVIDIA’s Multi-Instance GPU (MIG) [46] introduces hardware-level resource partitioning primitives that divide GPU resources, such as SMs, cache, and memory bandwidth, into multiple virtual GPU instances, each operating within an isolated environment. Besides vendor-provided solutions, TGS [68] integrates with Kubernetes to schedule workloads at the GPU kernel level, while REEF [28] extends the GPU driver to offer more fine-grained thread-level scheduling capabilities. The transparent design of these solutions facilitates broader adoption across applications. However, designing these solutions is often challenging due to the lack of application-level context.

3 GPU SHARING IN THE WILD

Despite the wide range of solutions, current GPU sharing systems are inadequate for practical deployment in DL clusters due to one or more of the following key limitations.

High integration and maintenance cost. Many existing GPU sharing systems are fundamentally intrusive in nature, necessitating extensive modifications to user code to be used in practice. This is particularly evident in application-level systems, where users must adapt their workloads to the specific programming interfaces proposed by these solutions. Such intrusive approaches not only burden the users, but also limit their flexibility in application development [31]. Similarly, runtime-level GPU sharing solutions that are designed as extensions of DL frameworks impose a significant burden on cluster managers to implement and maintain them across diverse frameworks used by the users. Consequently, deploying these solutions in practice often involves substantial integration and maintenance costs, especially given the rapidly evolving landscape of modern DL ecosystem.

On the other hand, while runtime-level GPU sharing solutions that integrate seamlessly with container orchestration systems or the GPU driver may appear non-intrusive, they often impose implicit requirements on user code to provide additional application contexts needed for making effective scheduling decisions. These include the ability to temporarily launch applications in a specialized profiling environment prior to full execution [58], or mandating applications to implement checkpoint and restart mechanisms to support live migration [42, 69]. However, enforcing such programming practices in large-scale, multi-tenant clusters is often impractical due to the operational complexity it introduces [67].

Table 1. Comparison of BERT inference latency against turnaround latency of different scheduling granularity for Whisper training on NVIDIA A100 GPU.

Inference time (BERT)	Turnaround latency (Whisper)			
	Iteration	Kernel	Block	Thread
3.93ms	~ 3s	~ 10ms	~ 304μs	~ 38μs

Lack of performance isolation guarantees. In production DL clusters, many inference tasks are governed by service-level agreements (SLAs), which impose strict stringent requirements on their response times, typically ranging from a few milliseconds to seconds [18]. However, current GPU sharing solutions often fall short in providing adequate performance isolation, leading to severe performance degradation for latency-critical inference tasks when co-located with other workloads [25]. For instance, NVIDIA MPS [1] is designed to eagerly schedule as many GPU kernels from co-located tasks as possible. While this design maximizes GPU utilization, it can cause significant queuing delays for executing kernels of the high-priority inference tasks. Our experiments (see Section 5) reveal that the tail latency of a co-located inference task under MPS can increase up to 20× compared to their standalone performance.

To effectively share GPU resources while ensuring performance isolation, a GPU sharing system must prioritize one task over another and rapidly switch between them according to their dynamic resource demands. The speed of context-switching between tasks, which we denote as the *turnaround latency*, is dictated by the scheduling granularity employed by the GPU sharing system. As described in Section 2, the scheduling granularity used by prior works primarily fall into four categories, from coarsest to finest: iteration-level (e.g. Salus [71]), kernel-level (e.g., TGS [68]), block-level (e.g. EffiSha [14]), and thread-level (e.g. REEF [28]). To illustrate the effect of scheduling granularity on the tail latency, we present a representative study in Table 1 where a latency-critical BERT [22] inference task is co-executed with a Whisper [51] model training workload on an NVIDIA A100 GPU [45]. In this experiment, we empirically estimate the turnaround latency for different scheduling granularities on the Whisper model and compare it to the response time of BERT inference. Since a BERT inference request takes only 3.93 ms, achieving strong performance isolation requires turnaround latencies to be in the sub-millisecond range. However, both iteration-level and kernel-level scheduling can only achieve turnaround latencies of 3 s and 10 ms, respectively. As a result, scheduling the Whisper training task at these granularities can introduce 2.5 – 760× higher overhead on BERT’s inference latency. In contrast, block-level and thread-level scheduling achieve microsecond-scale turnaround latencies of 304 μs and 38 μs, respectively, resulting in only mere 1 – 7% overhead on the latency.

From these results, we conclude that to achieve practical performance isolation guarantees, GPU sharing solutions must schedule resources at either the block-level or thread-level granularity. Unfortunately, existing solutions either do not meet these criteria (e.g., Salus [71], AntMan [70], TGS [68]) or prove ineffective in diverse production DL clusters due to their intrusive nature (e.g., EffiSha [14], Orion [58]) or lack of generalizability (e.g. REEF [28]). This brings us to the third limitation.

Reliance on narrowly applicable workload characteristics. Many state-of-the-art GPU sharing systems rely on specific workload characteristics in their scheduling strategies. For instance, AntMan and TGS provide runtime-level GPU sharing support but are limited to training workloads with predictable GPU kernel execution patterns. REEF, on the other hand, offers strong performance isolation for inference workloads; however, it requires all GPU kernels to be idempotent to support thread-level kernel scheduling strategies, meaning the kernels must always produce the same output for a given input, regardless of how many times it is executed. While these systems demonstrate benefits for certain classes of applications, their underlying assumptions are often not applicable to large-scale multi-tenant DL clusters that must support a diverse range of workloads with varying characteristics. More critically, prior studies [67] have revealed that DL clusters are often agnostic to the specific semantics of running workloads and extracting application-level context is largely infeasible. As a result, GPU sharing solutions that depend on specific task characteristics are not viable in many DL clusters.

In summary, we argue that a practical GPU sharing solution for large-scale DL clusters must address all three of the aforementioned limitations.

4 TALLY: SYSTEM DESIGN

To facilitate the practical deployment of GPU sharing in production DL clusters, we propose *Tally*, a novel non-intrusive GPU sharing system that provides robust performance isolation for high-priority tasks during shared execution with best-effort tasks across a diverse range of DL workloads. Tally is designed as a virtualization layer that operates transparently between applications and the GPU, intercepting device API calls and employing a task-agnostic scheduling algorithm to *prioritize* execution from *high-priority tasks* while *opportunistically scheduling* kernels from *best-effort tasks* during GPU idle cycles.

Tally ensures performance isolation for high-priority tasks by scheduling kernels from best-effort tasks at the block-level granularity using two scheduling primitives: slicing and preemption. Slicing divides a large kernel into numerous smaller segments, enabling more controlled scheduling, whereas preemption allows for the interruption of active kernels to expedite resource reallocation. Importantly, we

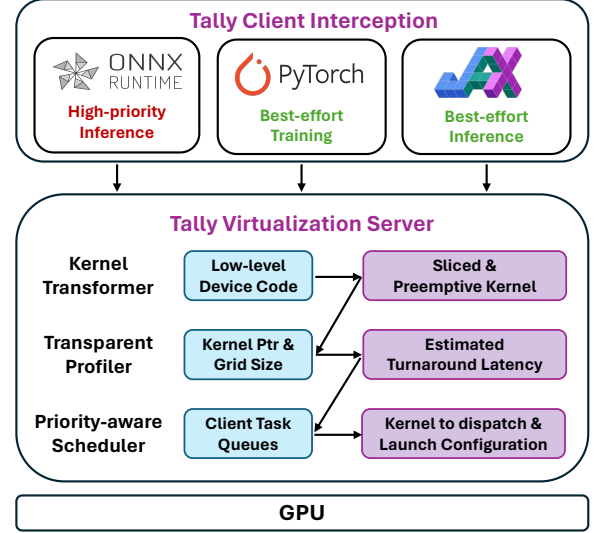


Figure 2. Tally architecture.

make two key observations that allow applying these two scheduling primitives in a task-agnostic and non-intrusive manner. Firstly, unlike thread-level scheduling, thread blocks within a GPU are guaranteed to operate independently, making block-level scheduling universally applicable across GPU kernels. Moreover, while thread-level scheduling can provide stronger performance isolation, we find that block-level scheduling is sufficient for most practical scenarios. Secondly, the slicing and preemption scheduling primitives can be applied on GPU kernels through a series of transformation passes on the kernel device code (PTX code in the case of NVIDIA GPUs) obtained through the interception of device API calls. Leveraging these insights, we integrate the slicing and preemption primitives to Tally’s scheduling algorithm to enable microsecond-scale turnaround latency, allowing Tally to promptly switch execution between high-priority and best-effort tasks and maintain the millisecond-scale latency requirements in many latency-critical tasks.

Figure 2 shows the high-level design of Tally, which follows a client-server architecture. On the application side, Tally’s client library intercepts device code and API calls initiated by client processes and forwards them to the Tally server, which is responsible for managing the actual device execution and resource scheduling. The objective of the Tally server is to facilitate priority-enforced concurrent execution of one high-priority task alongside multiple best-effort tasks. To achieve this, the Tally server incorporates three key components: a (1) *kernel transformer*, which leverages intercepted device code to convert kernels into slicing and preemptive-style kernels, a (2) *transparent profiler*, which profiles the performance of the transformed kernels under various launch configurations at runtime and estimates their turnaround latencies, and finally a (3) *priority-aware scheduler*, which integrates capabilities of the aforementioned modules and

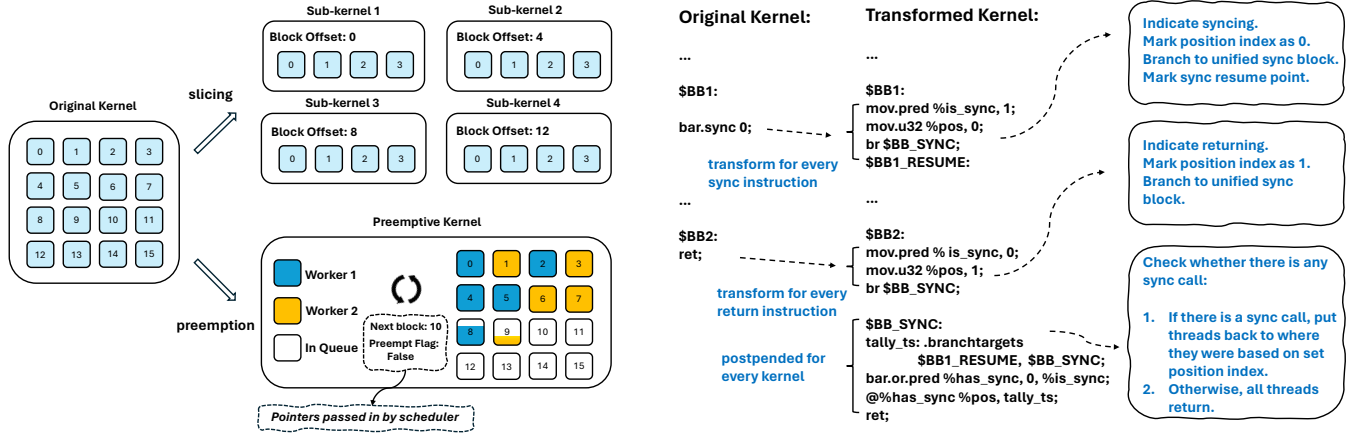


Figure 3. Kernel transformations: (a) slicing and preemption transformations (b) unified synchronization transformation.

strategically schedules GPU execution in a priority-enforced manner. In the following sections, we provide details regarding each of these components.

4.1 Kernel Transformations

We first illustrate how Tally supports the slicing and preemption primitives through a series of transformation passes on the kernel device code.

Slicing transformation. Slicing is a technique that facilitates fine-grained control over kernel execution by partitioning the thread blocks of a kernel into multiple sub-kernels. Figure 3(a) illustrates this concept, where a kernel originally composed of 16 thread blocks is divided into 4 sub-kernels, each containing 4 blocks. As thread blocks within a kernel are guaranteed to be independent, theoretically, this partitioning should preserve the original kernel’s functional semantics. However, naively launching the original kernel 4 times with 4 thread blocks each will yield incorrect results, as GPU threads rely on built-in variables such as *blockIdx* to determine their task assignment. Specifically, the *blockIdx* variable informs a thread of the index of the block it resides in within the kernel execution, allowing it to properly compute its assigned task index. In the original kernel, threads are exposed to *blockIdx* values ranging from 0 to 15 (as 16 blocks in total are launched). Yet, when each sub-kernel is launched, the threads are exposed to *blockIdx* values ranging from 0 to 3, causing all sub-kernels to execute the first 4 blocks of the kernel, which will lead to erroneous results. To address this issue, we modify a kernel’s device code to take in an additional offset variable in its parameters and substitute occurrences of *blockIdx* with the sum of the offset and the original *blockIdx*. This approach enables threads within a sub-kernel to identify their corresponding subset of blocks. For instance, sub-kernel 2 can reconstruct its corresponding block indices from 4 to 7 by adding a block offset of 4 to the new *blockIdx*. With this mechanism, the collective work done

by the sub-kernels can properly resemble the original kernel execution, while offering the scheduler flexible scheduling at the block-level granularity.

Preemption transformation. Tally introduces a transformation that makes kernels preemptible while preserving their functional semantics. The preemption transformation in Tally is inspired by the Persistent Thread Block (PTB) programming paradigm [27], which deviates from the traditional programming model where the number of thread blocks launched by a kernel is proportional to the amount of work it performs. Instead, the PTB model employs a flexible number of worker blocks to perform all the work. As an example, in Figure 3(a), rather than launching 16 thread blocks of the original kernel, the PTB model launches 2 worker blocks to process the 16 blocks in an iterative and dynamic manner. These workers continuously fetch and increment a global counter which represents the next task index. Similar to how sub-kernels in slicing utilize block offsets to recompute their block indices from the original kernel execution, workers in PTB use the task index to reconstruct the corresponding block indices for the current iteration. For example, in the illustrated case where thread blocks are structured in a 1-dimensional grid, a fetched task index of 8 indicates that the worker should perform the execution of the thread block with *blockIdx* 8. In the case of higher-dimensional grids, block indices can be computed from the integer task index by referring to the original kernel’s grid dimension.

A critical observation from the PTB model is that the progress of the kernel execution is effectively encapsulated in the global task index. Since the execution is now transformed into an iterative approach, kernels can potentially terminate early and later be resumed from the point at which they ended, using the task index. This can be achieved by introducing an additional boolean variable to the PTB kernels as a flag for signaling preemption. At the start of each iteration, workers check this flag to detect preemption signals,

thereby enabling block-level preemption within kernels. In Tally, we automatically convert kernels into this preemptive style by wrapping the original kernel function in the device code with an outer loop using control flow logic. For instance, all return statements are transformed into branch instructions that direct the execution flow to the start of the iteration. This ensures that each worker block will continue execution until all tasks are processed or a preemption signal is received. Furthermore, a synchronization call is introduced at the end of each iteration to make sure that threads within a worker block process a single block at a time, preventing potential race conditions and maintaining correctness of computation.

Unified synchronization transformation. In the preemption transformation, return statements are substituted with branching statements that redirect threads to synchronize at the start of the iteration. However, this modification introduces a risk of threads within the same block calling synchronization at different places within the kernel. For instance, while some threads may arrive at one of the kernel's original synchronization points, others may be redirected to the newly introduced synchronization point due to early returning. Such divergence in synchronization is an undefined behavior in the GPU programming model and, in practice, results in infinite kernel stalls. This issue makes the preemption transformation unsafe to be applied on all kernels.

To address this problem, Tally introduces a propositional kernel transformation pass which we denote as the *unified synchronization transformation*. The algorithm for this transformation is depicted in Figure 3(b). As illustrated, this transformation modifies all synchronization and return instructions by redirecting them to a unified synchronization point that checks whether all threads have reached a return state. If this condition is met, all threads return simultaneously; otherwise, threads from synchronization points loop back to their previous execution positions, while those returning are held until all threads align. This modification guarantees that threads within a block return synchronously. Thus, when the kernel undergoes the preemption transformation, the risk of divergence in thread block synchronization due to early returns is eliminated, thus resolving the issue of infinite stalls. Through this propositional transformation stage, the preemptive transformation can be applied automatically and safely to all kernels.

4.2 Priority-aware Scheduling

We now demonstrate how Tally's priority-aware scheduler leverages the aforementioned slicing and preemption primitives alongside the transparent profiler to enforce priority during the scheduling of concurrent workloads. The algorithm of the scheduler, as outlined in Figure 4, follows an opportunistic strategy: it prioritizes the immediate dispatch of high-priority kernels upon their arrival (lines 14 to 20) and schedules low-priority kernels to execute only when the

```

1 def launch_and_profile(kernel):
2     candidates = get_candidate_configs(kernel)
3     for cfg in candidates:
4         est_turnaround = lookup_measurement(kernel, cfg)
5         if est_turnaround is None:
6             est_turnaround = profile(kernel, cfg)
7         if has_launched():
8             return
9     set_launch_config(kernel, candidates,
10                      bound=TUNEAROUND_LATENCY_BOUND)
11 def scheduler():
12     while True:
13         for client in sort_by_priority(clients):
14             if client.is_high_priority():
15                 kernel = client.fetch_next_kernel()
16                 if kernel is not None:
17                     preempt_best_effort_kernel()
18                     kernel.launch(Config::DEFAULT)
19                 if client.has_kernel_running():
20                     break
21             else:
22                 kernel = client.get_curr_ex_kernel()
23                 if kernel is None:
24                     kernel = client.fetch_next_kernel()
25                     cfg = lookup_launch_config(kernel)
26                     if cfg is not None:
27                         kernel.launch(cfg)
28                 else:
29                     launch_and_profile(kernel)
30             elif kernel.running():
31                 pass
32             elif kernel.stopped():
33                 kernel.resume()

```

Figure 4. Tally's priority-aware scheduling algorithm.

high-priority job becomes inactive (lines 21 to 33). Due to the reactive nature of this algorithm, incoming high-priority kernels may need to wait for the completion of currently executing low-priority kernels, potentially introducing delays to the execution of high-priority tasks. To mitigate such queuing delays, each best-effort kernel is either launched as multiple sub-kernels one at a time, or in its preemptive form, such that it can be preempted in the event of a high-priority kernel arriving during its execution.

The primary challenge of this strategy lies in determining the appropriate launch configuration for each best-effort kernel, which includes deciding whether to employ slicing or preemption and setting the corresponding parameters for each method. First, due to the inherent difference in their implementation, each of the two primitives exhibits distinct performance trade-offs. While slicing is a lightweight transformation, it can incur overhead due to multiple kernel launches. Conversely, the preemption kernel requires only a single launch, but the additional synchronization and control flow can introduce noticeable overhead to certain kernels. As a result, their relative performance across different kernels varies considerably, posing challenges for the scheduler

in choosing between the two. Moreover, for each primitive, the scheduler need to determine the corresponding parameter, which includes the number of slices for slicing and the number of workers for preemption. These decisions are crucial as they significantly affect the performance behavior of the concurrent workloads. For instance, in the context of preemptive scheduling, allocating fewer workers can expedite the preemption process but can potentially slow down the execution of best-effort tasks, thereby diminishing the overall benefits of GPU sharing.

To strike a balance between ensuring robust performance isolation for high-priority tasks and maintaining acceptable throughput for low-priority tasks, Tally’s priority-aware scheduler employs a profile-guided resource provisioning policy. This policy leverages runtime performance metrics to determine the optimal launch configuration for each best-effort kernel through a search process. As depicted in Figure 4, lines 2 to 10, for each kernel, the scheduler generates a set of candidate configurations encompassing both slicing and preemption. For preemption, potential configurations are derived by considering all possible multiples of the number of SMs that fit with the thread constraints. Whereas slicing may set the degree of slicing based on having each slice operating on different percentages of the total blocks. These candidate configurations are then assessed by Tally’s transparent profiler.

To evaluate the performance of candidate configurations, Tally employs the *turnaround latency* metric, as introduced in Section 3. This metric represents the estimated time needed for a best-effort kernel to release occupied resources. Calculating the turnaround latency for a sliced kernel is relatively straightforward, which can be done by simply measuring the completion time of a single kernel slice. In contrast, determining the turnaround latency for preemptive kernels is more challenging due to their use of dynamic worker blocks. To address this challenge, we employ a heuristic approach that approximates the turnaround latency by dividing the total execution time of the kernel by the number of blocks allocated to each worker, as expressed by:

$$\text{turnaround_latency} = \frac{\text{kernel_latency} \cdot \text{worker_blocks}}{\text{total_blocks}}. \quad (1)$$

This formula approximates the latency of waiting for each worker to complete the current block of execution. After profiling each configuration, the scheduler selects the one that achieves the optimal performance while complying with a predefined turnaround latency threshold, which serves as a configurable parameter to balance the performance trade-offs between high and low-priority workloads. Empirically, we find that a default value of 0.0316 ms can effectively render robust performance isolation while producing competitive system throughput (details in Section 5.6).

Notably, Tally’s profiling process accounts for variations in kernel inputs by profiling each unique work configuration

(block and grid dimensions) separately for every kernel. For measurement stability, profiling results are averaged across many runs (e.g., 10). Since profiling operates on a per-kernel basis, once measurements for a kernel are collected, they can be reused throughout the execution, thus introducing negligible overhead to overall execution.

4.3 Non-intrusive Virtualization Layer

Finally, we present details on the implementation of Tally’s virtualization layer. On the application-side, Tally utilizes the LD_PRELOAD mechanism in Linux to intercept device API calls, redirecting them to a surrogate function that mirrors the API’s original signature. These intercepted API calls are sent to the server, which handles the actual device execution. The challenge in this process is enabling the server, which operates in a separate address space, to execute client kernel functions. The solution leverages a critical insight: GPU applications register device code with the GPU driver at the start of execution. By intercepting this process, the server gains access to the device code, including kernel binaries and PTX code, which we use at the server-side to apply kernel transformations and recompile to executable GPU kernels.

A notable concern with this interception and forwarding approach is the potential communication overhead, which could impact application performance. To mitigate this, we have implemented two critical optimizations. First, we utilize efficient shared memory channels for client-server communication, which significantly reduces the overhead by avoiding context switches during message passing. Second, we observe that the need to forward many frequent device API calls, such as *cudaGetDevice* in NVIDIA, can be avoided by maintaining local states of the execution context on the client side. This approach effectively minimizes the frequency of API call forwarding between the client and server, further reducing the communication overhead. These strategies collectively enable Tally to operate transparently with near-native performance across all the concurrent DL tasks.

5 EVALUATION

5.1 Methodology

Workloads. To evaluate Tally, we prepare a benchmark suite consisting of six training and six inference workloads as listed in Table 2, covering a diverse range of widely used DL model architectures, including CNNs (e.g., ResNet50 [29]), transformer-based LLMs (e.g., Llama-2 [62]), and diffusion models (e.g., Stable Diffusion [54]). Training workloads are implemented using PyTorch [49], a broadly adopted DL framework in the research community. For inference workloads, we use high-performance inference engines such as ONNX Runtime [21], TorchInductor [5], and Hidet [23]. Some kernels in the workloads are sourced from proprietary libraries such as cuBLAS [43], which restrict device code interception. To enable block-level transformations, Tally

automatically replaces these kernels at runtime with CUTLASS [32] alternatives that offer similar performance. Inspired by prior DL serving benchmarks [8, 28, 38], we simulate inference workloads using the Microsoft Azure Function Trace 2021 (MAF2) [77], a dataset of Azure serverless function invocations that has been extensively repurposed for DL serving research. We extract the trace of the most frequently invoked function and adapt it to match specific traffic loads based on the inference latency of the models. We define *load* as the percentage of time an inference service is actively serving requests.

Metrics. We use the 99th percentile latency as the primary metric for assessing the performance of inference tasks. For both training and inference tasks, we measure their *throughput*, defined as the number of samples processed per unit of time. To evaluate the overall performance of concurrent task execution, we calculate *system throughput* by summing the normalized throughput of all concurrent workloads.

Baselines. We compare the performance of Tally against four state-of-the-art non-intrusive GPU sharing solutions: (i) Time-Slicing, (ii) MPS, (iii) MPS-Priority, and (iv) TGS. All these systems perform scheduling at the kernel-level granularity. Specifically, Time-Slicing and MPS are the proprietary concurrency mechanisms provided by NVIDIA GPUs. MPS-Priority refers to MPS with the client priority control feature enabled, which allows assigning priority values to workloads for prioritized resource allocation. TGS is a GPU sharing system specifically designed for DL workloads that leverages adaptive rate control mechanisms to achieve performance isolation across concurrent workloads.

Experimental setup. All experiments are conducted on an AWS EC2 p4d.24xlarge instance, a production-grade GPU server widely deployed in DL clusters. It is equipped with a 96-core Intel Xeon Platinum 8275CL CPU, 1152 GB of DRAM, and 8 NVIDIA A100 SXM GPUs, each with 40 GB of memory. The server runs on Ubuntu 20.04, and is configured with CUDA 12.2, PyTorch 2.2.0, ONNX Runtime GPU 1.17.0, and Hidet 0.3.0.

5.2 End-to-end Results

We first analyze Tally’s end-to-end performance in efficiently sharing GPU resources among multiple workloads while guaranteeing performance isolation for high-priority inference tasks. To this end, we measure the 99th-percentile latencies of the inference tasks and the overall system throughput achieved by Tally when executed concurrently with a best-effort training workload. First, we compare the latency metrics against the state-of-the-art non-intrusive GPU sharing systems, namely Time-Slicing, MPS, MPS-Priority, and TGS, as well as against the ideal scenario of running the inference task in isolation. Figure 5 presents the latency and throughput measurements for all inference-training combinations from the benchmark suite, with inference tasks simulated

Training Model	Dataset	# Params	Throughput
ResNet50 [29]	ImageNet [20]	25.6M	1.0 it/s
PointNet [50]	ShapeNet [12]	3.5M	40.0 it/s
BERT [22]	SQuAD [53]	110M	1.8 it/s
GPT2-Large [52]	Wikitext2 [39]	774M	3.3 it/s
PEGASUS [75]	XSum [40]	568M	2.9 it/s
Whisper-v3 [51]	LibriSpeech [48]	1.5B	0.3 it/s

Inference Model	Engine	# Params	Latency
ResNet50 [29]	Hidet [23]	25.6M	1.37 ms
BERT [22]	ONNX RT [21]	110M	3.93 ms
YOLOv6m [37]	TorchInductor [5]	34.9M	17.5 ms
Llama-2-7b [62]	ONNX RT [21]	7B	1.9 s
Stable Diffusion [54]	TorchInductor [5]	983M	2.5 s
GPT-Neo [9]	TorchInductor [5]	2.7B	3.6 s

Table 2. Training and inference benchmarks

using the MAF2 trace with a fixed 50% load. The results demonstrate that Time-Slicing, MPS, MPS-Priority, and TGS exhibit significantly higher overhead compared to the ideal latency, with an average increase of 252.3%, 345.0%, 195.5%, and 188.9%, respectively. The suboptimal inference latencies exhibited by Time-Slicing and MPS can be attributed to their priority-agnostic design, which focuses solely on maximizing system utilization. MPS-Priority and TGS, on the other hand, exhibit improved latencies by prioritizing kernel execution of high-priority tasks over best-effort tasks. However, their coarse-grained kernel-level scheduling strategies fail to effectively mitigate co-execution interference. In stark contrast, Tally consistently maintains tail latencies nearly identical to the ideal latencies of the high-priority workloads, with an average percentage increase of mere 7.2%. This stronger performance isolation capability can be attributed to the block-level scheduling strategy that Tally employs.

In addition to latency, we also compare the overall system throughput of Tally against the baseline GPU sharing systems. Tally demonstrates competitive performance relative to the baselines, attaining an average of 105.2%, 83.6%, 80.6%, and 80.3% of the system throughput achieved by the four baseline systems, respectively. This shows the ability of Tally’s priority-aware scheduler to opportunistically schedule best-effort kernels while preserving latency requirements of high-priority tasks in shared execution environments.

Furthermore, we observe that MPS, MPS-Priority, and TGS exhibit substantial variation in performance degradation for high-priority inference tasks across different workload combinations. For instance, TGS experiences performance degradation ranging from 15.6% to 751.7%. This variation in latency reveals the sensitivity of these systems to workload characteristics, such as the differences in kernel duration across various tasks. As these systems schedule at the kernel

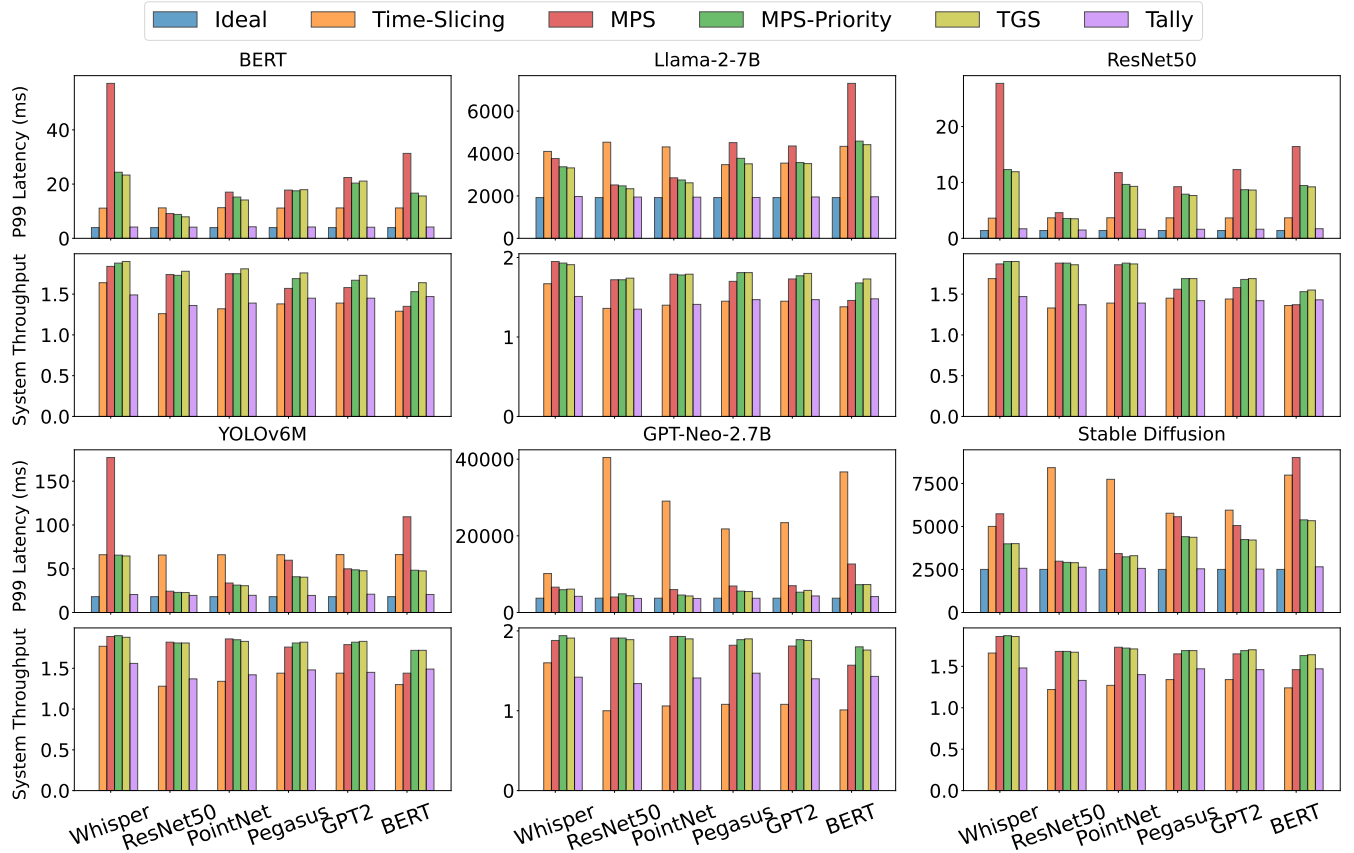


Figure 5. Analysis of 99th-percentile latency and system throughput across different high-priority inference and best-effort training workload combinations under various GPU sharing mechanisms.

level, their performance deteriorates more severely for workloads with longer kernels. In contrast, Tally’s block-level scheduling demonstrates robust performance against variations in kernel duration, effectively limiting the performance degradation of inference tasks to under 10% across 70% of the workload combinations (worst-case 23% on ResNet50).

5.3 Traffic Load Sensitivity Analysis

We next evaluate the sensitivity of Tally’s performance isolation capabilities to varying traffic loads. For this experiment, we use BERT and Llama-2-7B as the inference tasks representative of low-latency (3.93 ms) and long-latency (1.9 s) workloads. Each of them is paired with three distinct training workloads: BERT, GPT2, and Whisper, chosen for their observed tendency to cause significant interference. Figure 6a (1) and (2) present a comparative analysis of the 99th-percentile latency for the inference tasks under Tally and TGS relative to the original latency as a function of GPU idle time (calculated as $100\% - \text{load}$) ranging from 10% to 90%. The results demonstrate that both inference tasks under Tally exhibit latencies indistinguishable from their original performance consistently across varying loads. In contrast, TGS

incurs latency slowdowns of up to 5.8× and 2.3× for BERT and Llama-2-7B, respectively. These findings demonstrate the robustness of Tally’s performance isolation capabilities for high-priority inference tasks, regardless of traffic load or the characteristics of the paired training workload.

We further analyze the relationship between Tally’s achievable throughput and traffic load. Figure 6a (3) and (4) illustrate the system throughput under Tally and TGS as a function of idle time. Both systems exhibit an upward trend in system throughput as the percentage of idle time increases, indicating their ability to leverage available resources for improved performance. During periods of high traffic load, there is a noticeable difference between the two systems. For instance, at 10% idle time, the system throughput under Tally is 1.19, whereas TGS achieves 1.65. This difference is attributed to Tally’s strict priority enforcement policy, which executes best-effort tasks only when the high-priority task is inactive, sacrificing potential throughput gains for the preservation of the high-priority task’s latency. In contrast, TGS achieves higher throughput but at the cost of significant slowdowns in tail latency, as demonstrated earlier. Nevertheless, as idle time increases, the throughput difference

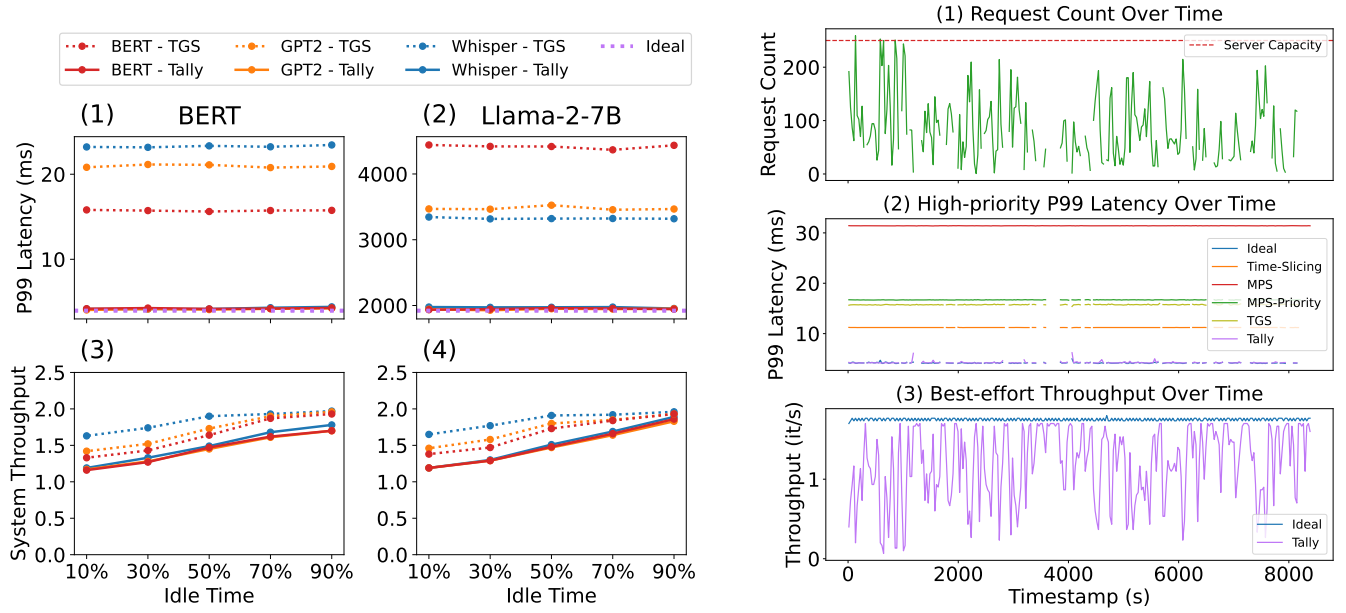


Figure 6. (a) Latency and throughput for high-priority BERT and Llama2-7B inference co-located with BERT, GPT-2, and Whisper training under Tally and TGS across different traffic loads. (b) Time-series visualization of user traffic, tail latency, and throughput over time for BERT inference co-located with BERT training across different GPU sharing systems.

between Tally and TGS diminishes. This convergence indicates that for scenarios where heavy underutilization is observed, Tally not only offers reliable performance isolation guarantees but also achieves competitive throughput compared to other GPU sharing systems.

To illustrate the adaptive nature of Tally to varying traffic load, in Figure 6b, we plot the real-time traffic trace to a BERT inference task (a condensed version of the original MAF2 trace), and measure the real-time tail latency and throughput when co-located with another BERT training task. Figure 6b illustrates (1) the real-time traffic to a BERT inference task, (2) 99th percentile latency of the inference server achieved by different GPU sharing systems and (3) the throughput of the co-located best-effort BERT training achieved by Tally compared to the original throughput of the training task. As shown in Figure 6b (2), the tail latency under Tally, represented by the purple line, closely aligns with the ideal latency depicted by the blue line, with negligible deviation throughout the simulation period. Whereas alternative approaches such as Time-Slicing, MPS, MPS-Priority, and TGS result in substantial slowdowns in latency as depicted. Additionally, Figure 6b (3) shows Tally’s ability to opportunistically adjust the throughput of the best-effort task in correspondence with the fluctuating traffic load to the high-priority inference task, as discussed in Section 4. As a result, Tally preserves over 68% of the training task’s original throughput simultaneously running the high-priority inference workload on the same GPU without affecting its latency requirements.

5.4 Scalability with Number of Workloads

To demonstrate Tally’s ability to support multiple best-effort workloads while preserving the performance of the co-located high-priority job, we extend our experiments beyond the two-job scenario. We designed an experiment with a set of identical ResNet50 inference workloads, each operating at a low traffic load of 10% and one of these workloads designated as the high-priority online inference task while the rest as best-effort offline inference tasks. We measure 99th-percentile latency of the high-priority task and the aggregated system throughput, measured in number of requests processed per minute, as we scale up the number of best-effort tasks. The results in Figure 7(a), reveal that the latency of the high-priority task remains consistent throughout the spectrum of scaling from 1 to 10 workloads. At the same time, the system throughput exhibits a steady increase as the number of workloads grows, until the GPU saturates at 8 concurrent best-effort workloads. This shows that Tally can be particularly useful in scenarios where numerous jobs exhibit extremely low utilization, as Tally can efficiently pack these workloads together to maximize system throughput without compromising the performance of high-priority tasks.

5.5 Performance Decomposition

We analyze the importance of Tally’s priority-aware scheduling algorithm and block-level transformations for performance isolation. Figure 7(b) compares the 99th-percentile latency of a high-priority BERT model when co-located with

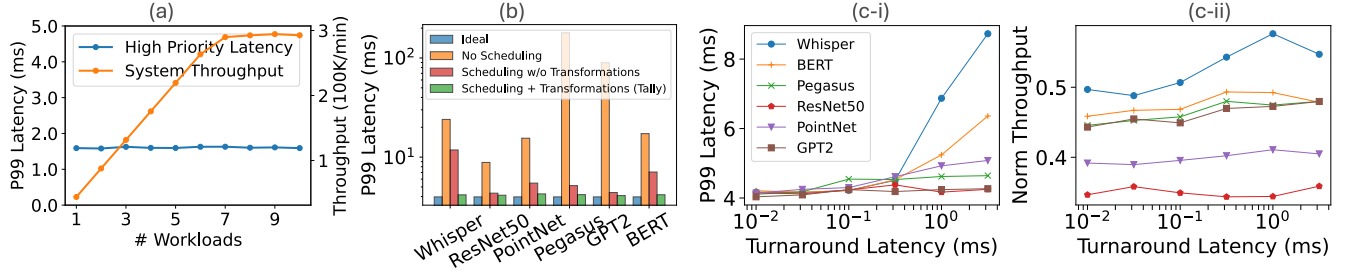


Figure 7. (a) Scalability performance of Tally with respect to number of concurrent workloads. (b) Performance decomposition of Tally for BERT inference. (c) Latency and throughput under different turnaround latency threshold settings.

training workloads from the benchmark suite. *Ideal* denotes the original latency of the task executed in isolation. *No-scheduling* refers to the indiscriminate dispatching of kernels from both workloads upon arrival. *Scheduling w/o Transformations* employs Tally’s priority-aware scheduling without kernel transformations, while *Scheduling with Transformations* represents the full implementation of Tally, utilizing slicing and preemption mechanisms for block-level scheduling. The results reveal that No-scheduling incurs substantial latency overhead across all 6 job combinations, with the most severe case exhibiting a 30× slowdown when co-located with Whisper. This significant performance degradation is attributable to the absence of priority enforcement, resulting in considerable co-execution interference. The integration of Tally’s priority-aware kernel-level scheduling strategy somewhat mitigates this overhead. For ResNet50 and GPT2, this approach yields latencies close to ideal performance, with 8.0% and 9.8% slowdown respectively. However, for workloads such as Whisper and BERT, the latency slowdown can still reach up to 10×. This difference in slowdown is attributed to the variation in kernel duration distribution. For instance, 99.3% of kernels in ResNet50 complete in less than 0.1 ms, whereas 5.6% of kernels in Whisper exceed the duration of an entire BERT inference (3.93 ms). These prolonged kernels significantly delay the execution of high-priority kernels. Notably, with kernel transformation enabled, i.e., with full Tally, the latency overhead becomes negligible across all six training workloads, resulting in an average of 4.0% slowdown (6.2% in the worst case). This shows the necessity of block-level scheduling in maintaining robust performance isolation guarantees.

5.6 Turnaround Latency Analysis

We examine the impact of the turnaround latency threshold (delineated in Section 4) by evaluating the tail latency of the BERT inference workload and the normalized throughput of the co-located training job on six models across six distinct threshold values, ranging from 0.01 ms to 10 ms. The results, depicted in Figure 7 (c-i) and (c-ii), show that higher turnaround latency thresholds generally correspond to increased

tail latency for the inference workload with only slight increase in the throughput. Based on these empirical results, we find that a turnaround latency threshold of 0.0316 ms provides the best balance between latency and throughput, which is the default setting we use throughout experiments.

5.7 Overhead Analysis

Virtualization. We evaluate Tally’s virtualization overhead by measuring the throughput of each workload in the benchmark suite and comparing it to their baseline performance. Results indicate that virtualization incurs an average overhead of only 1%, proving minimal performance impact.

Kernel transformation. We evaluate the overhead of Tally’s kernel transformation by profiling 10K kernels from best-effort workloads (including same kernel with varying work dimensions). Our analysis reveals an average overhead of 25%. Notably, this overhead is exclusive to best-effort kernels. The performance of high-priority jobs remain unaffected.

Profiling. Tally profiles each new kernel from best-effort tasks to determine its optimal launch configuration. Once collected, these measurements are reused throughout the execution. Thus, Tally’s profiling stage for a job generally completes within minutes. This overhead is negligible given that training workloads often run for hours to days.

6 DISCUSSION

Comparison with MIG. NVIDIA MIG [46] enables hardware-level GPU partitioning into isolated instances, providing robust performance isolation for concurrent workloads. However, MIG’s coarse-grained partitioning mechanism inherently limits the maximum resource allocatable to individual workloads, which can result in potential performance degradation of high-priority tasks. This limitation stems from MIG’s requirement for configuring resource partitions prior to task execution; any alteration to these partitions necessitates the termination of all running processes, rendering it impractical in production environments where job interruption is typically prohibited. Even when reconfiguration is feasible, this process can incur significant overhead: resetting

partitions takes several seconds via command-line instructions, and applications require additional time from seconds to minutes to resume execution. As a result, MIG struggles to promptly adapt resource allocations to meet the dynamic demands of DL workloads, potentially leading to compromised performance of high-priority workloads. Additionally, MIG’s static allocation scheme can result in suboptimal resource utilization, as idle cycles within one partition cannot be dynamically reallocated to others. These limitations prevent MIG from simultaneously achieving high utilization while maintaining the performance of high-priority tasks. In contrast, Tally effectively addresses these challenges by dynamically allocating resources based on real-time resource demands, allowing high-priority tasks to fully utilize the GPU when active while reallocating idle cycles to best-effort tasks, thereby ensuring both high utilization and effective performance isolation.

Generalization to CUDA extensions. While the standard GPU programming model enforces the independence of thread blocks within a kernel (as explained in Section 2), recent CUDA extensions, such as *Thread Block Clusters* [16] and *Cooperative Groups* [17], introduce support for inter-block dependencies under certain constraints. We illustrate how Tally can reliably detect and handle these cases. (1) Thread Block Clusters organizes thread blocks into clusters, enabling intra-cluster communication via hardware support. These kernels can be identified by special flags passed at kernel launch. In such cases, Tally applies slicing and preemption at the cluster level, rather than at the thread block level. This ensures execution correctness while enabling more fine-grained scheduling. (2) Cooperative Groups introduces new thread grouping mechanisms that extend the capabilities of the standard CUDA programming model. Among them is inter-block synchronization, which enables coordinated execution across thread blocks. Kernels using this feature are launched through the `cudaLaunchCooperativeKernel` API, making them easily identifiable. Notably, inter-block synchronization requires that all thread blocks within a kernel must be simultaneously scheduled on the GPU. For these kernels, Tally refrains from applying block-level scheduling to ensure compliance with the concurrent execution requirements. In our evaluation, none of the workloads employ inter-block cooperative groups, suggesting that the absence of block-level scheduling for these kernels has a negligible impact on Tally’s overall performance isolation guarantees.

7 RELATED WORK

Many GPU sharing systems have been proposed with the goal of maximizing throughput. NVIDIA’s Time-Slicing [25] and MPS [1], for example, offer mechanisms to share GPU resources in temporal and spatial manners, respectively. Moreover, several studies [19, 61, 66] have explored co-execution

of kernels with complementary resource demands (e.g., compute vs. memory) to exploit resource heterogeneity in GPUs. Systems like Kernelet [80] and Slate [4] schedule such co-execution dynamically at runtime, while approaches like Kernel Fusion [64], HFUSE [36], Tacker [79], and ISPA [78] employ source-to-source transformations to fuse kernels during the compilation phase. Despite their efficacy in improving resource utilization, these systems inherently lack support for performance isolation, as they prioritize overall system efficiency over preservation of performance of individual workloads, making them unsuitable for production environments where tasks must adhere to strict SLAs.

To address the need for performance isolation, solutions like Effisha [14] and GPES [81] utilize block-level preemption techniques to mitigate interference during concurrent execution. However, these approaches require access to the application source code, whereas Tally operates non-intrusively. Other solutions, such as AntMan [70] and TGS [68], leverage the iterative nature of DL training to adaptively limit the resources of low-priority tasks based on performance feedback from high-priority jobs. Meanwhile, REEF [28] implements a reset-based scheduling mechanism that achieves microsecond-scale kernel preemption latency on idempotent kernels. Yet, these works rely on specific task characteristics and cannot generalize to the wide variety of DL workloads. In contrast, Tally supports broad compatibility using task-agnostic scheduling and kernel transformations. Furthermore, Orion [58] enhances performance isolation via kernel-level scheduling with consideration on each kernel’s compute and memory requirements. However, it requires offline profiling of workloads, which is challenging to accomplish in cluster settings. In comparison, Tally employs transparent profiling for priority-aware scheduling.

8 CONCLUSION

In this paper, we highlight the limitations of current GPU sharing systems and their inability to meet the diverse requirements of modern large-scale DL clusters. To overcome these challenges, we design Tally, a novel non-intrusive GPU sharing mechanism that offers robust performance isolation while maintaining compatibility with a wide range of DL workloads. Our evaluation shows that Tally can effectively ensure the performance standards of latency-critical tasks during shared execution, incurring a mere 7.2% overhead on average while offering competitive system throughput.

Acknowledgments

We thank the anonymous reviewers and our shepherd, Tim Rogers, for their valuable feedback. This research was supported in part by the Innovation JELF Grant, NSERC Discovery Grant, AWS Machine Learning Research Award (MLRA), Facebook Faculty Research Award, Google Scholar Research Award, and VMware Early Career Faculty Grant.

A Artifact Appendix

A.1 Abstract

We provide the source code of Tally along with scripts to reproduce experimental results presented in the evaluation section. This appendix includes instructions for generating plots similar to Figure 5 and Figure 7(b). Dockerfiles are provided to set up the required runtime environment. To expedite artifact evaluation, a pre-built image is available, containing the fully configured environment, pre-compiled deep learning models, and datasets as specified in Table 2. The experiments require an x86-64 Linux host with at least 85 GB RAM, 200 GB of free disk space, and an NVIDIA A100 GPU (40 GB). For consistent performance, we recommend using a Google Cloud a2-highgpu-1g instance.

A.2 Artifact check-list (meta-information)

- **Run-time environment:** Docker.
- **Compilation:** Pre-compiled within a Docker image.
- **Data set:** MAF2 trace and datasets as listed in Table 2.
- **Hardware:** A 12-core CPU with 85 GB RAM and an NVIDIA A100 GPU (40 GB).
- **Metrics:** 99th percentile latency and throughput.
- **Experiments:** End-to-end evaluation (Section 5.2) and performance decomposition (Section 5.5). For faster evaluation, the default setup generates results only for co-executing BERT and Llama inference tasks alongside all training workloads. Instructions for running the full set of inference workloads can be found in Section A.7.
- **Output:** Plots similar to Figure 5 and Figure 7(b).
- **How much disk space required (approximately)?:** Approximately 200 GB.
- **How much time is needed to prepare workflow (approximately)?:** About 10 minutes to download the Docker image.
- **How much time is needed to complete experiments (approximately)?:** Approximately 6 hours for BERT and Llama tasks; 18 hours for the full end-to-end evaluation.
- **Publicly available?:** Yes.
- **Code licenses (if publicly available)?:** MIT License.

A.3 Description

A.3.1 How to access. The artifact is available for download on GitHub: <https://github.com/tally-project/tally-bench>.

A.3.2 Hardware dependencies. Requires an x86-64 Linux host with at least 85 GB of RAM, 200 GB of free disk space, and an NVIDIA A100 GPU (40 GB). We recommend a Google Cloud a2-highgpu-1g instance for consistent performance.

A.3.3 Software dependencies. Docker installation is required as all experiments are configured to run within the provided Docker image.

A.3.4 Data sets. The data sets used in the experiments are listed in Table 2 and are pre-downloaded as part of the Docker image.

A.3.5 Models. The models used in these experiments are listed in Table 2.

A.4 Installation

1. Clone the Github repository.

```
$ git clone
  https://github.com/tally-project/tally-bench.git
$ cd tally-bench
```

2. Pull the Docker image.

```
$ docker pull wzhaol8/tally:bench
```

A.5 Experiment workflow

1. Create the results directory and launch the Docker container.

```
$ mkdir tally_results
$ docker run -it --shm-size=64g --gpus 0 -v
  ${PWD}/tally_results:/home/tally-bench/tally_results
  wzhaol8/tally:bench /bin/bash
```

2. Run the following commands to run the experiments.

(On host machine)

```
$ sudo nvidia-smi -i 0 -c DEFAULT
```

(In docker container)

```
$ ./scripts/run_bench.sh > tally_results/b1.log 2>&1
```

(On host machine)

```
$ sudo nvidia-smi -i 0 -c EXCLUSIVE_PROCESS
```

(In docker container)

```
$ ./scripts/run_bench.sh > tally_results/b2.log 2>&1
```

A.6 Evaluation and expected results

1. Parse results into CSV files.

(On host or in container)

```
$ python3 ./scripts/parse_results.py
```

2. Generate plots.

(On host or in container)

```
$ python3 ./scripts/plot_results_micro.py
```

3. Plots similar to Figure 5 and Figure 7(b) can be found in the tally_results directory.

A.7 Experiment customization

To run the full benchmark, pass the `--run-full-benchmark` flag to `run_bench.sh`. Use the `scripts/plot_results.py` script to generate the plots. Additional scripts for running other experiments and producing figures included in the paper are available in the `scripts` directory.

References

- [1] Multi-process service. Available online, 2024. <https://docs.nvidia.com/deploy/mps/>.
- [2] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. Tensorflow: A system for large-scale machine learning. In *12th USENIX*

- Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265–283, 2016.
- [3] Taleb Alashkar, Songyao Jiang, Shuyang Wang, and Yun Fu. Examples-rules guided deep neural network for makeup recommendation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 31, 2017.
 - [4] Tyler Allen, Xizhou Feng, and Rong Ge. Slate: Enabling workload-aware efficient multiprocessing for modern gpgpus. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 252–261, 2019.
 - [5] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarikar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhresch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '24, page 929–947, New York, NY, USA, 2024. Association for Computing Machinery.
 - [6] Andrew Audibert, Yang Chen, Dan Graur, Ana Klimovic, Jiri Šimša, and Chandramohan A. Thekkath. tf.data service: A case for disaggregating ml input data processing. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*, SoCC '23, page 358–375, New York, NY, USA, 2023. Association for Computing Machinery.
 - [7] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.
 - [8] Anirban Bhattacharjee, Ajay Dev Chhokra, Zhuangwei Kang, Hongyang Sun, Aniruddha Gokhale, and Gabor Karsai. Barista: Efficient and scalable serverless serving system for deep learning prediction services. In *2019 IEEE International Conference on Cloud Engineering (IC2E)*, pages 23–33. IEEE, 2019.
 - [9] Sid Black, Gao Leo, Phil Wang, Connor Leahy, and Stella Biderman. GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow, March 2021. If you use this software, please cite it using these metadata.
 - [10] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
 - [11] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
 - [12] Angel X Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. Shapenet: An information-rich 3d model repository. *arXiv preprint arXiv:1512.03012*, 2015.
 - [13] Chao Chen, Chris Porter, and Santosh Pande. Case: a compiler-assisted scheduling framework for multi-gpu systems. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPOPP '22, page 17–31, New York, NY, USA, 2022. Association for Computing Machinery.
 - [14] Guoyang Chen, Yue Zhao, Xipeng Shen, and Huiyang Zhou. Effisha: A software framework for enabling efficient preemptive scheduling of gpu. *SIGPLAN Not.*, 52(8):3–16, jan 2017.
 - [15] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele. The cityscapes dataset for semantic urban scene understanding. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3213–3223, Los Alamitos, CA, USA, jun 2016. IEEE Computer Society.
 - [16] NVIDIA Corporation. *CUDA C++ Programming Guide*. NVIDIA Corporation, 2024. Accessed: 2024-10-26.
 - [17] NVIDIA Corporation. *CUDA C++ Programming Guide*. NVIDIA Corporation, 2024. Accessed: 2024-10-26.
 - [18] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A Low-Latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 613–627, Boston, MA, March 2017. USENIX Association.
 - [19] Weihao Cui, Mengze Wei, Quan Chen, Xiaoxin Tang, Jingwen Leng, Li Li, and Mingyi Guo. Ebird: Elastic batch for improving responsiveness and throughput of deep learning services. In *2019 IEEE 37th International Conference on Computer Design (ICCD)*, pages 497–505, 2019.
 - [20] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
 - [21] ONNX Runtime developers. Onnx runtime. <https://onnxruntime.ai/>, 2021. Version: x.y.z.
 - [22] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Tamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics.
 - [23] Yaoyao Ding, Cody Hao Yu, Bojian Zheng, Yizhi Liu, Yida Wang, and Gennady Pekhimenko. Hidet: Task-mapping programming paradigm for deep learning tensor programs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS 2023, page 370–384, New York, NY, USA, 2023. Association for Computing Machinery.
 - [24] Doug Eadline. Nvidia h100: Are 550,000 gpus enough for this year?, Aug 2023.
 - [25] Guin Gilman and Robert J. Walls. Characterizing concurrency mechanisms for nvidia gpus under deep learning workloads. *Performance Evaluation*, 151:102234, 2021.
 - [26] Inc. Google. Kubernetes: Production-grade container orchestration. <https://kubernetes.io>, 2014. Accessed: 2024-06-24.
 - [27] Kshitij Gupta, Jeff A. Stuart, and John D. Owens. A study of persistent threads style gpu programming for gpgpu workloads. In *2012 Innovative Parallel Computing (InPar)*, pages 1–14, 2012.
 - [28] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 539–558, Carlsbad, CA, July 2022. USENIX Association.
 - [29] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
 - [30] Qinghao Hu, Peng Sun, Shengren Yan, Yonggang Wen, and Tianwei Zhang. Characterization and prediction of deep learning workloads in large-scale gpu datacenters. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2021.
 - [31] Qinghao Hu, Meng Zhang, Peng Sun, Yonggang Wen, and Tianwei Zhang. Lucid: A non-intrusive, scalable and interpretable scheduler

- for deep learning training jobs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS 2023, page 457–472, New York, NY, USA, 2023. Association for Computing Machinery.
- [32] Andrew Kerr, Duane Merrill, Julien Demouth, and John Tran. *CUTLASS: Fast Linear Algebra in CUDA C++*. NVIDIA Corporation, December 2017. Version 2.10.
- [33] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012.
- [34] Kevin Lee, Kevin Lee, Adi Gangidi, and Mathew Oldham. Building meta’s genai infrastructure, May 2024.
- [35] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, Online, July 2020. Association for Computational Linguistics.
- [36] Ao Li, Bojian Zheng, Gennady Pekhimenko, and Fan Long. Automatic horizontal fusion for gpu kernels. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 14–27. IEEE, 2022.
- [37] Chuyi Li, Lulu Li, Hongliang Jiang, Kaiheng Weng, Yifei Geng, Liang Li, Zaidan Ke, Qingyuan Li, Meng Cheng, Weiqiang Nie, et al. Yolov6: A single-stage object detection framework for industrial applications. *arXiv preprint arXiv:2209.02976*, 2022.
- [38] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 663–679, Boston, MA, July 2023. USENIX Association.
- [39] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.
- [40] Shashi Narayan, Shay B. Cohen, and Mirella Lapata. Don’t give me the details, just the summary! Topic-aware convolutional neural networks for extreme summarization. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Brussels, Belgium, 2018.
- [41] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. {Heterogeneity-Aware} cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498, 2020.
- [42] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-Aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498. USENIX Association, November 2020.
- [43] NVIDIA Corporation. *Basic Linear Algebra on NVIDIA GPUs*, 2024. Version 12.0.
- [44] NVIDIA Corporation. *CUDA C++ Programming Guide*, version 12.5 edition, June 2024. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [45] NVIDIA Corporation. NVIDIA A100 Tensor Core GPU, 2024. Accessed: 2024-12-13.
- [46] NVIDIA Corporation. *NVIDIA Multi-Instance GPU*, June 2024. <https://www.nvidia.com/en-us/technologies/multi-instance-gpu/>.
- [47] OpenAI. Chatgpt. Available online, 2024. <https://chat.openai.com/chat>.
- [48] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: An asr corpus based on public domain audio books. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5206–5210, 2015.
- [49] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [50] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.
- [51] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning*, pages 28492–28518. PMLR, 2023.
- [52] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [53] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In Jian Su, Kevin Duh, and Xavier Carreras, editors, *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas, November 2016. Association for Computational Linguistics.
- [54] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.
- [55] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [56] Dharma Shukla, Muthian Sivathanu, Srinidhi Viswanatha, Bhargav Gulavani, Rimma Nehme, Amey Agrawal, Chen Chen, Nipun Kwatra, Ramachandran Ramjee, Pankaj Sharma, et al. Singularity: Planet-scale, preemptive and elastic scheduling of ai workloads. *arXiv preprint arXiv:2202.07848*, 2022.
- [57] David Silver, Aja Huang, Christopher J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–503, 2016.
- [58] Foteini Strati, Xianzhe Ma, and Ana Klimovic. Orion: Interference-aware, fine-grained gpu sharing for ml applications. In *Proceedings of the Nineteenth European Conference on Computer Systems*, EuroSys ’24, page 1075–1092, New York, NY, USA, 2024. Association for Computing Machinery.
- [59] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2*, NIPS’14, page 3104–3112, Cambridge, MA, USA, 2014. MIT Press.
- [60] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, 2015.
- [61] Xiaodan Serina Tan, Pavel Golikov, Nandita Vijaykumar, and Gennady Pekhimenko. Gpupool: A holistic approach to fine-grained gpu sharing in the cloud. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, PACT ’22, page 317–332, New York, NY, USA, 2023. Association for Computing Machinery.
- [62] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal

- Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [63] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- [64] Guibin Wang, YiSong Lin, and Wei Yi. Kernel fusion: An effective method for better power efficiency on multithreaded gpu. pages 344–350, 2010.
- [65] Shang Wang, Peiming Yang, Yuxuan Zheng, Xin Li, and Gennady Pekhimenko. Horizontally fused training array: An effective hardware utilization squeezer for training novel deep learning models. *Proceedings of Machine Learning and Systems*, 3:599–623, 2021.
- [66] Zhenning Wang, Jun Yang, Rami Melhem, Bruce Childers, Youtao Zhang, and Minyi Guo. Simultaneous multikernel gpu: Multi-tasking throughput processors via fine-grained sharing. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 358–369, 2016.
- [67] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. MLaaS in the wild: Workload analysis and scheduling in Large-Scale heterogeneous GPU clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 945–960, Renton, WA, April 2022. USENIX Association.
- [68] Bingyang Wu, Zili Zhang, Zhihao Bai, Xuanzhe Liu, and Xin Jin. Transparent GPU sharing in container clouds for deep learning workloads. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 69–85, Boston, MA, April 2023. USENIX Association.
- [69] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610, Carlsbad, CA, October 2018. USENIX Association.
- [70] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. AntMan: Dynamic scaling on GPU clusters for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 533–548. USENIX Association, November 2020.
- [71] Peifeng Yu and Mosharaf Chowdhury. Fine-grained gpu sharing primitives for deep learning applications. In I. Dhillon, D. Papailiopoulos, and V. Sze, editors, *Proceedings of Machine Learning and Systems*, volume 2, pages 98–111, 2020.
- [72] Seongjun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang, and Hyunwoo J. Kim. *Graph transformer networks*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [73] Ekim Yurtsever, Jacob Lambert, Alexander Carballo, and Kazuya Takeda. A survey of autonomous driving: Common practices and emerging technologies. *IEEE access*, 8:58443–58469, 2020.
- [74] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan. MARK: Exploiting cloud services for Cost-Effective, SLO-Aware machine learning inference serving. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 1049–1062, Renton, WA, July 2019. USENIX Association.
- [75] Jingqing Zhang, Yao Zhao, Mohammad Saleh, and Peter Liu. Pegasus: Pre-training with extracted gap-sentences for abstractive summarization. In *International conference on machine learning*, pages 11328–11339. PMLR, 2020.
- [76] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. Deep learning based recommender system: A survey and new perspectives. *ACM Comput. Surv.*, 52(1), feb 2019.
- [77] Yanqi Zhang, Íñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. Faster and cheaper serverless computing on harvested resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, page 724–739, New York, NY, USA, 2021. Association for Computing Machinery.
- [78] Han Zhao, Weihao Cui, Quan Chen, and Minyi Guo. Ispa: Exploiting intra-sm parallelism in gpus via fine-grained resource management. *IEEE Trans. Comput.*, 72(5):1473–1487, may 2023.
- [79] Han Zhao, Weihao Cui, Quan Chen, Youtao Zhang, Yanchao Lu, Chao Li, Jingwen Leng, and Minyi Guo. Tacker: Tensor-cuda core kernel fusion for improving the gpu utilization while ensuring qos. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 800–813, 2022.
- [80] Jianlong Zhong and Bingsheng He. Kernelet: High-throughput gpu kernel executions with dynamic slicing and scheduling. *IEEE Transactions on Parallel and Distributed Systems*, 25(6):1522–1532, 2013.
- [81] Husheng Zhou, Guangmo Tong, and Cong Liu. Gpes: a preemptive execution system for gpgpu computing. In *21st IEEE Real-Time and Embedded Technology and Applications Symposium*, pages 87–97, 2015.